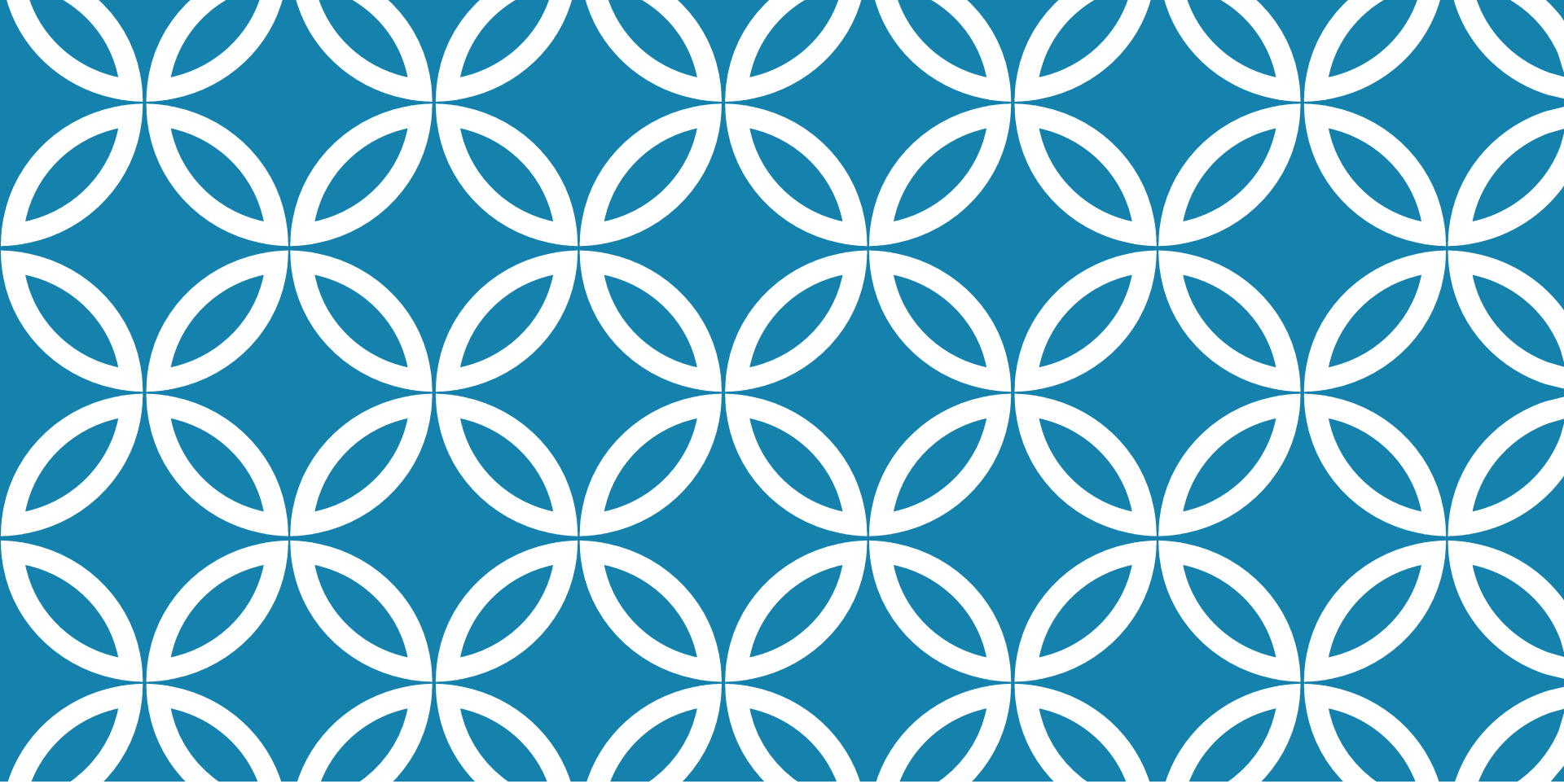


COMPONENT BASED THINKING

Lecture 3

LAST TIME

GRASP

Package Design principles

- Coupling
- Cohesion

CONTENT

Architecture characteristics

- What are they?
- How do we **identify** them?
- How do we **measure** them?
- How do we **achieve** them?

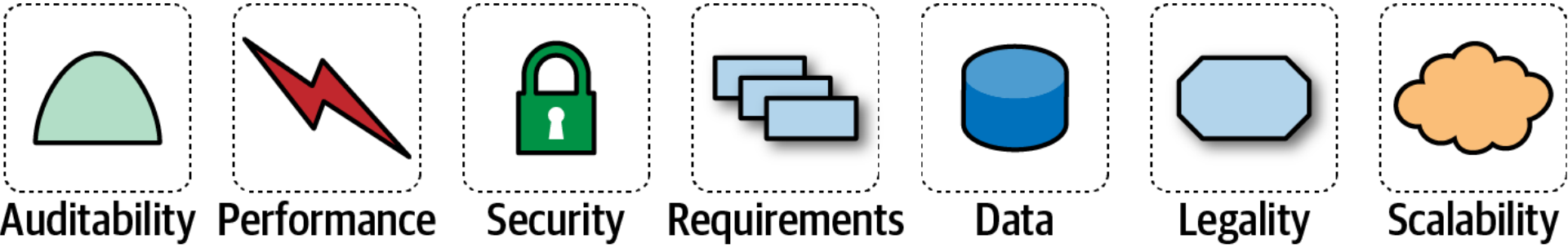
REFERENCES

Fundamentals of Software Architecture, Mark Richards, Neal Ford, 2020 - Chapters 4,5,6,8

ARCHITECTURE CHARACTERISTICS

Specifications for a software solution:

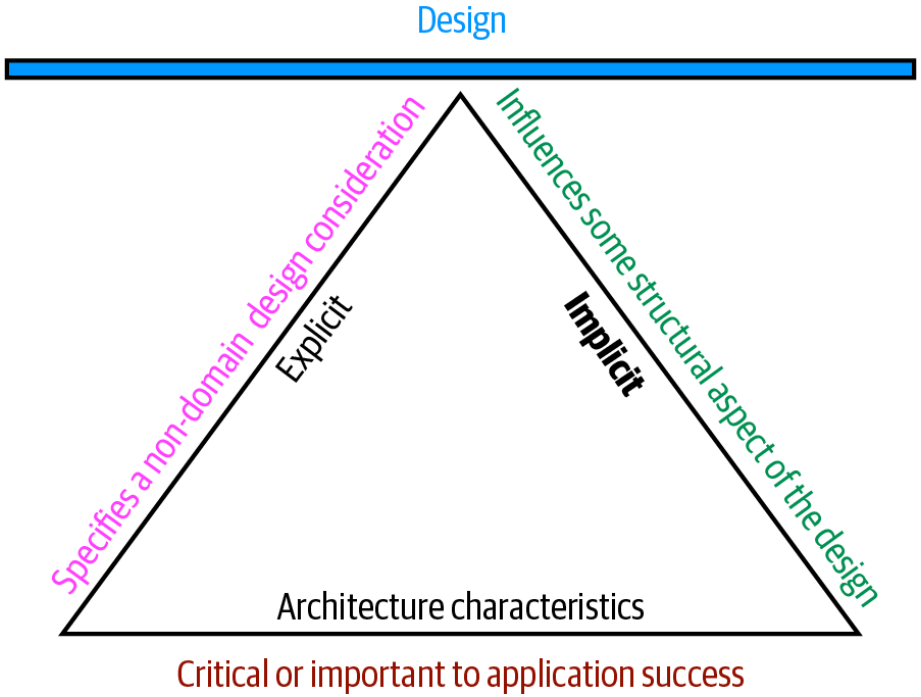
- Functional requirements (aka business requirements)
- Architectural characteristics (aka non-functional requirements, quality attributes)



WHAT ARE ARCHITECTURE CHARACTERISTICS?

Criteria:

- Is critical or important to application success
- Specifies a nondomain design consideration
- Influences some structural aspect of the design



OPERATIONAL ARCHITECTURE CHARACTERISTICS

Term	Definition
Availability	How long the system will need to be available (measured as uptime percentage (ex. 99.99%)).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.

OPERATIONAL ARCHITECTURE CHARACTERISTICS

Term	Definition
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

STRUCTURAL ARCHITECTURE CHARACTERISTICS

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/reuse	Ability to leverage common components across multiple products.

STRUCTURAL ARCHITECTURE CHARACTERISTICS

Term	Definition
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

CROSS-CUTTING ARCHITECTURE CHARACTERISTICS

Term	Definition
Accessibility	Access to all your users, including those with disabilities like colorblindness or hearing loss.
Archivability	Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).
Legal	What legislative constraints is the system operating in (data protection, Sarbanes Oxley, GDPR, etc.)? What reservation rights does the company require? Any regulations regarding the way the application is to be built or deployed?

CROSS-CUTTING ARCHITECTURE CHARACTERISTICS

Term	Definition
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability/achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.

HOW TO IDENTIFY ARCHITECTURE CHARACTERISTICS?

Extracting Architecture Characteristics from **Domain Concerns:**

- Keep the list as short as possible
- Anti-patterns: generic architecture that supports ALL the architecture characteristics!

Domain concern	**Architecture characteristics**
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

EXTRACTING ARCHITECTURE CHARACTERISTICS FROM REQUIREMENTS

- explicit statements in **requirements** documents (ex. expected numbers of users and scale)
- Inherent domain knowledge (ex. student registration system)
- Architecture katas (https://www.architecturalkatas.com/)
  - **Description** - The overall domain problem the system is trying to solve
  - **Users** - The expected number and/or types of users of the system
  - **Requirements** - Domain/domain-level requirements, as an architect might expect from domain users/domain experts
  - **Additional context** - Many of the considerations an architect must make aren't **explicitly** expressed in requirements but rather by **implicit** knowledge of the problem domain

SILICON SANDWICH EXAMPLE

Description

A national sandwich shop wants to enable online ordering (in addition to its current call-in service).

Users

Thousands, perhaps one day millions

Requirements

- Users will place their order, then be given a time to pick up their sandwich and directions to the shop (must integrate with several external mapping services that include traffic information)
- If the shop offers a delivery service, dispatch the driver with the sandwich to the user

SILICON SANDWICH EXAMPLE CONTINUED

- ….
- Mobile-device accessibility
- Offer national daily promotions/specials
- Offer local daily promotions/specials
- Accept payment online, in person, or upon delivery

Additional context

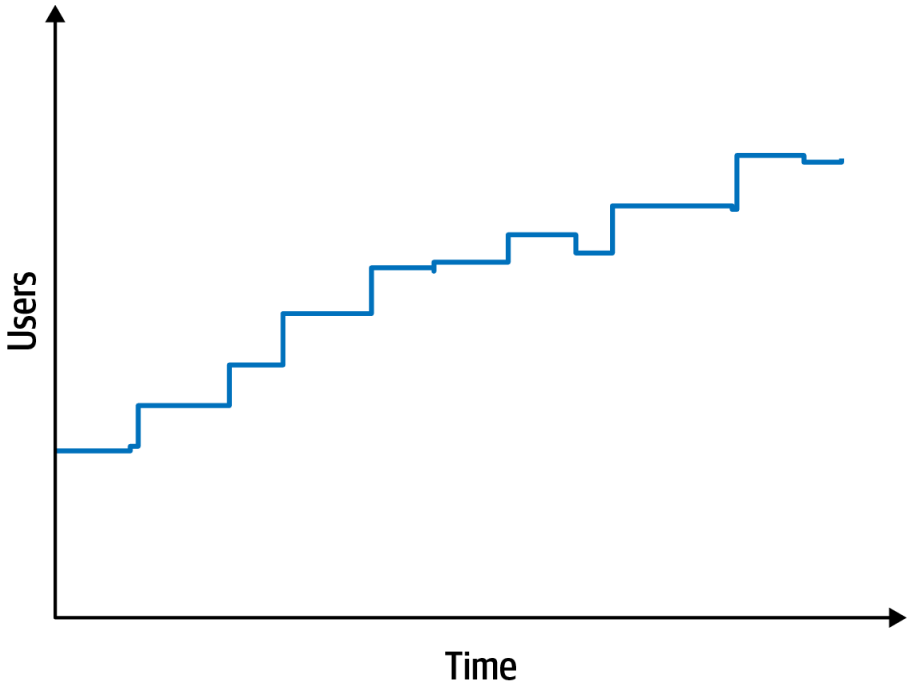
- Sandwich shops are franchised, each with a different owner
- Parent company has near-future plans to expand overseas
- Corporate goal is to hire inexpensive labor to maximize profit

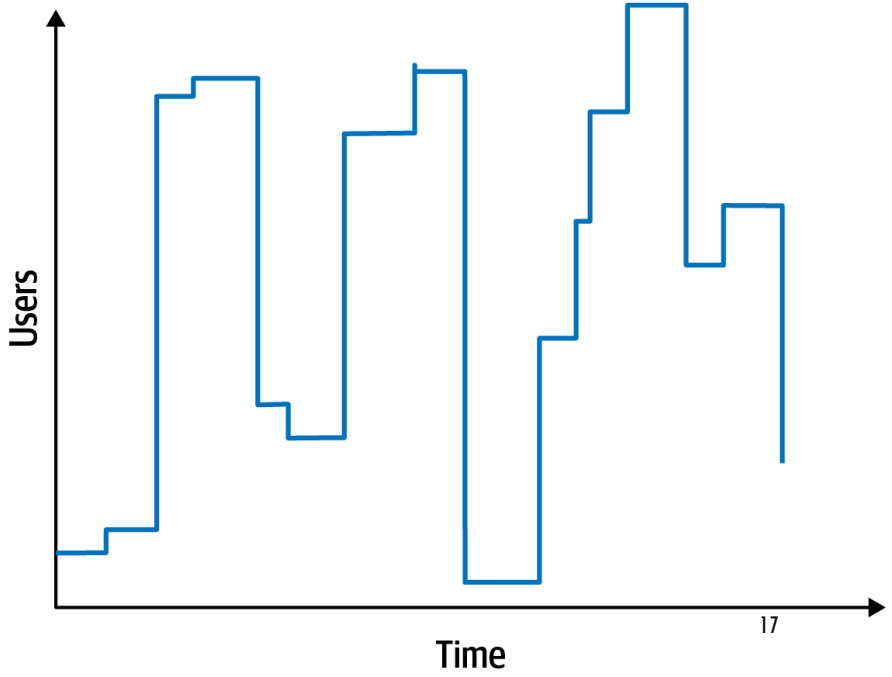
SILICON SANDWICH EXAMPLE

Explicit characteristics

- "number of users: currently thousands, perhaps one day millions" => **scalability + elasticity**

- "Users will place their order, …(must provide the option to integrate with external mapping services that include traffic information)." => **reliability**





SILICON SANDWICH EXAMPLE

Explicit characteristics

- "Mobile-device accessibility." => **design** decisions on page load time and other mobile-sensitive characteristics
- "Offer national and local daily promotions/specials" => **customizability**
- "Parent company has near-future plans to expand overseas"=> **internationalization**
- Performance? (Homework)

SILICON SANDWICH EXAMPLE

Implicit characteristics

- Availability: making sure users can access the sandwich site
- Reliability: making sure the site stays up during interactions
- Security

Architectural decisions

- Customizability: Microkernel architecture vs. another architecture (ex. Template method design pattern) ?
- are there good reasons, such as performance and coupling, not to implement a microkernel architecture?
- are other desirable architecture characteristics more difficult in one design versus the other?
- how much would it cost to support all the architecture characteristics in each design versus pattern?

There are no wrong answers in architecture, only expensive ones.

HOW TO MEASURE AND MANAGE ARCHITECTURE CHARACTERISTICS?

Problems with measuring architecture characteristics:

- They are not physical (ex. deployability?)
- Wildly varying semantic (ex. performance)
- Composition (ex. agility = modularity + deployability + testability)

⇒Need for objective definitions

Operational measures

- Performance:
  - Average response time for certain requests?
  - Are outliers acceptable?
  - First contentful paint
  - First CPU idle
- Scalability

STRUCTURAL MEASURES

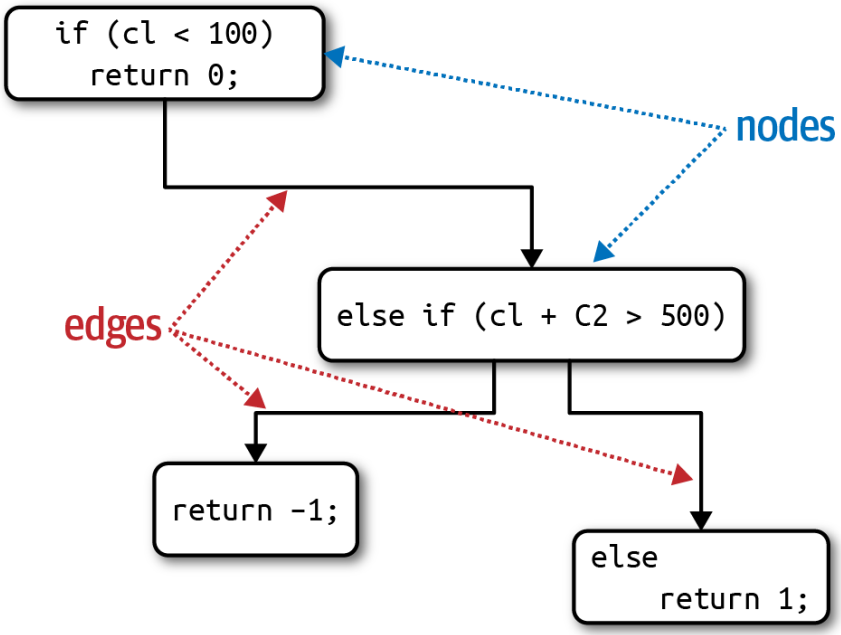
- Code complexity: cyclomatic complexity

CC = E-N+2P, where N = nodes (lines of code), E = edges (decisions), P = connected components (calls to other methods)

Example

```
public void decision(int c1, int
c2) {
    if (c1 < 100)
        return 0;
    else if (c1 + C2 > 500)
       return 1;
    else
      return -1;
}
```

Ideally CC <= 5



PROCESS MEASURES

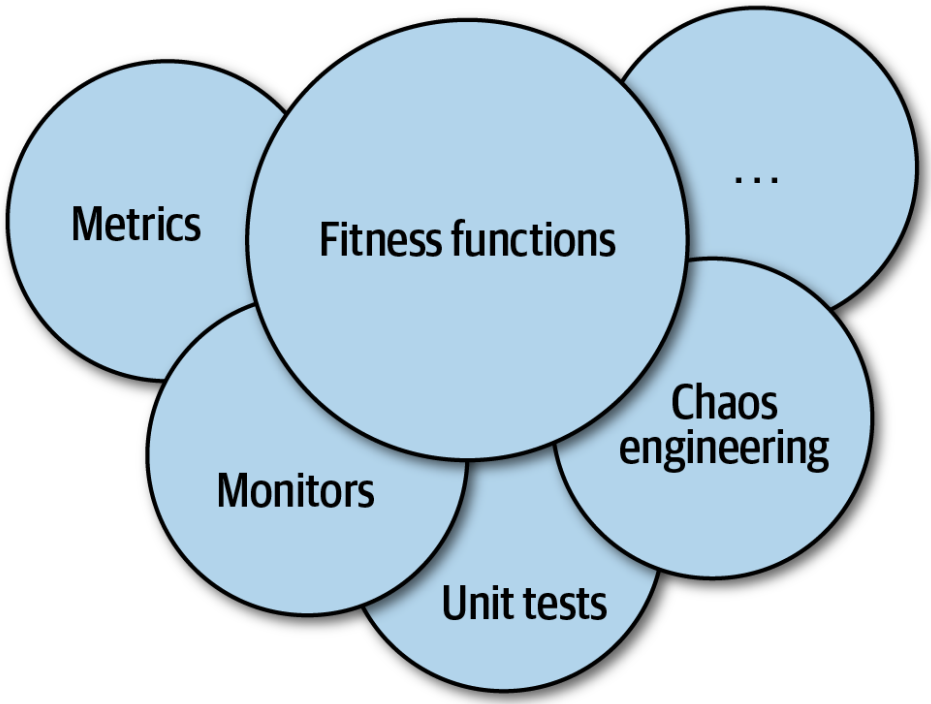
- Agility = testability + deployability
- Testability:
  - Code coverage
- Deployability
  - Percentage of successful to failed deployments
  - Time deployments take
  - Issues/bugs raised by deployments

HOW DO WE ACHIEVE ARCHITECTURE CHARACTERISTICS?

Governing architecture characteristics

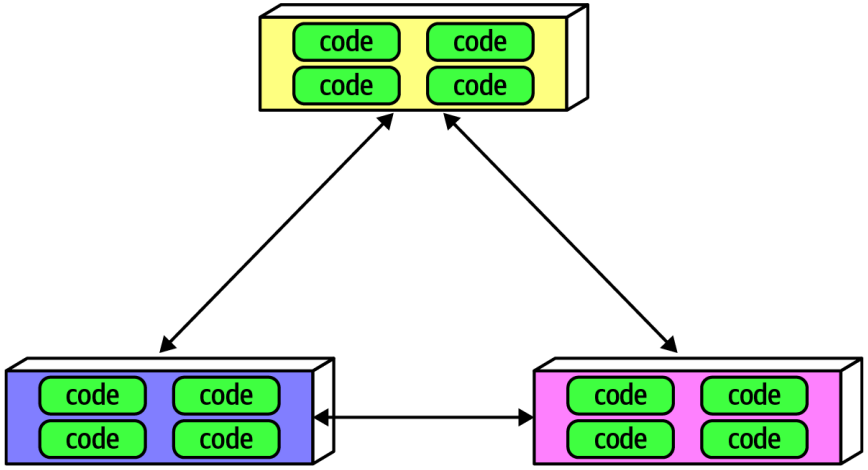
- Automated, continuous integration

=> DevOps



Architecture Fitness functions: Any mechanism that provides an **objective integrity assessment** of some architecture characteristic or combination of architecture characteristics

FITNESS FUNCTIONS



Cyclic dependencies

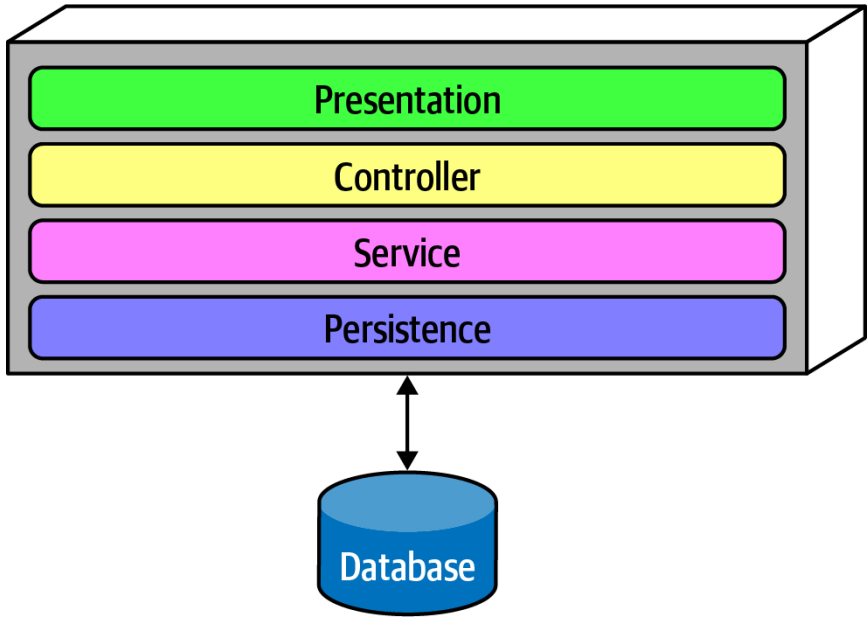
```
public class CycleTest {
    private JDepend jdepend;
    @BeforeEach
    void init() {
        jdepend = new JDepend();
        jdepend.addDirectory("/path/to/project/persistence/classes");
        jdepend.addDirectory("/path/to/project/web/classes");
        jdepend.addDirectory("/path/to/project/thirdpartyjars");
    }
    @Test
    void testAllPackages() {
        Collection packages = jdepend.analyze();
        assertEquals("Cycles exist", false,jdepend.containsCycles());
    }}
```

FITNESS FUNCTIONS

Distance from the main diagonal

```
@Test
void AllPackages() {
    double ideal = 0.0;
    double tolerance = 0.5; // project-dependent
    Collection packages = jdepend.analyze();
    Iterator iter = packages.iterator();
    while (iter.hasNext()) {
      JavaPackage p = (JavaPackage)iter.next();
      assertEquals("Distance exceeded: " + p.getName(),
          ideal, p.distance(), tolerance);
    }
}
```

FITNESS FUNCTIONS



Respecting the layers

ArchUnit fitness function

```
layeredArchitecture()
   .layer("Controller").definedBy("..controller..")
   .layer("Service").definedBy("..service..")
   .layer("Persistence").definedBy("..persistence..")

.whereLayer("Controller").mayNotBeAccessedByAnyLayer
()
.whereLayer("Service").mayOnlyBeAccessedByLayers("Co
ntroller")
.whereLayer("Persistence").mayOnlyBeAccessedByLayers
("Service")
```

HOMEWORK

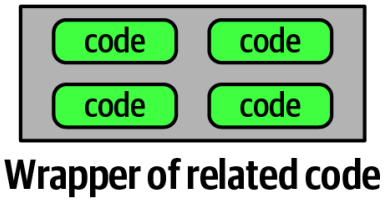
Read in Chapter 6 about Netflix's Chaos Monkey and the attendant Simian Army

HOW TO ACHIEVE ARCHITECTURE CHARACTERISTICS?

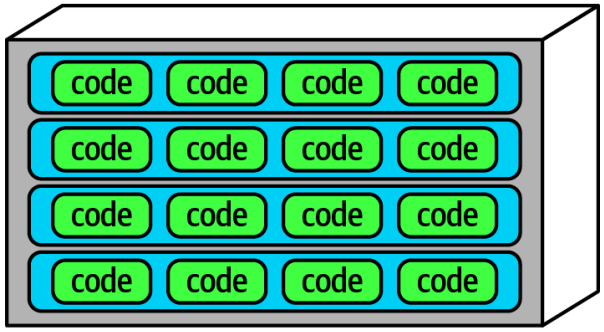
Component based thinking

Component = physical packaging of modules (i.e. jar, dll, gem)

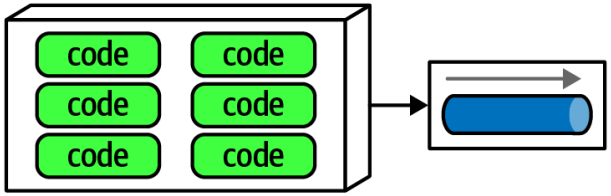
Component Scope:



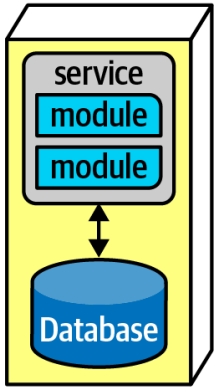
Wrapper of related code



Layer or subsystem



Event processor



Distributed service

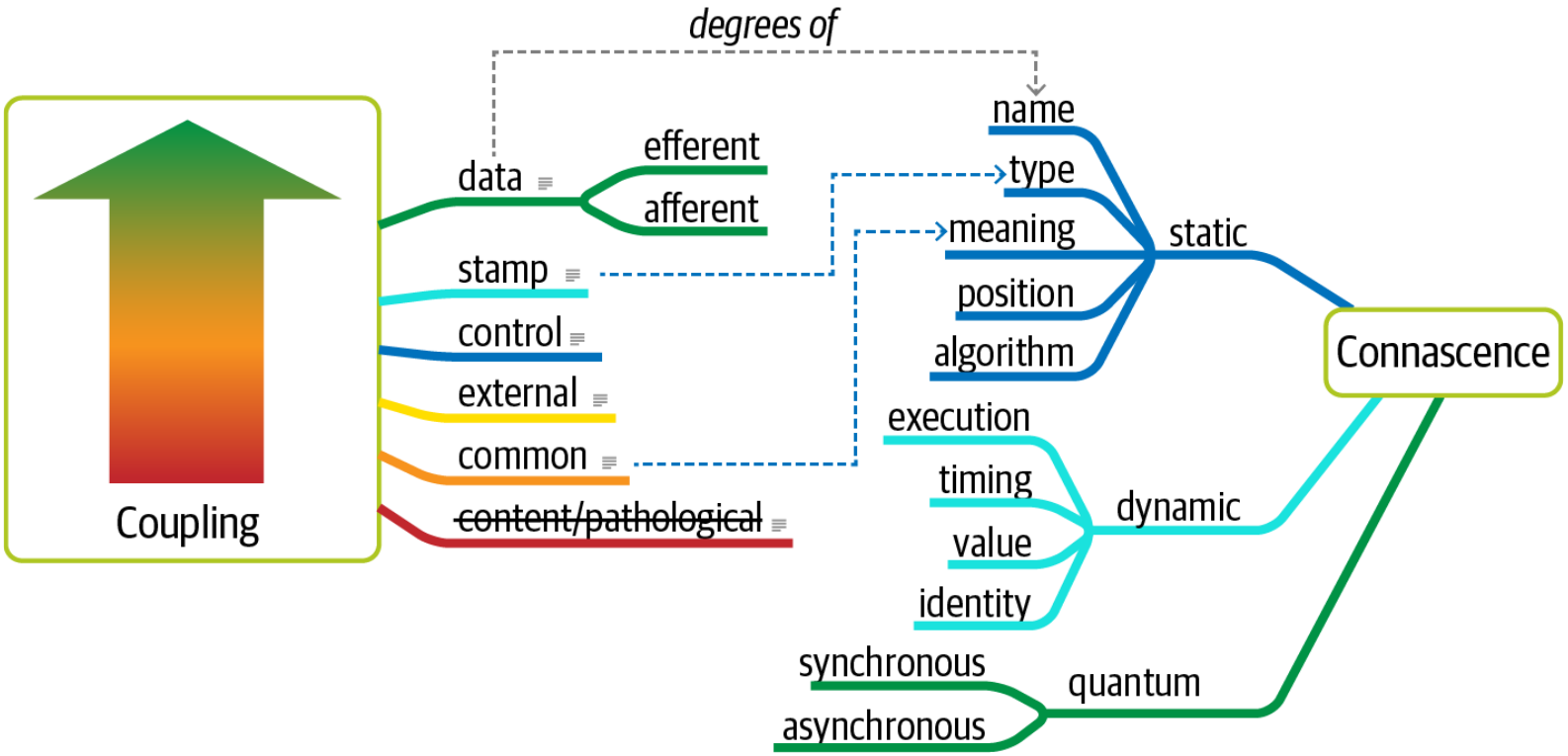
ARCHITECTURAL QUANTUM/QUANTA

- **Coupling**
- **Connascence:** Two components are connascent if a change in one would require the other to be modified in order to maintain the overall correctness of the system
  - Static: discoverable via static code analysis

  Ex. two services in a microservices architecture share the same class definition of some class
  - Dynamic: concerning runtime behavior
    - Synchronous
    - Asynchronous

ARCHITECTURAL QUANTUM/QUANTA

- **Architecture quantum:** An independently deployable artifact with high functional cohesion and synchronous connascence

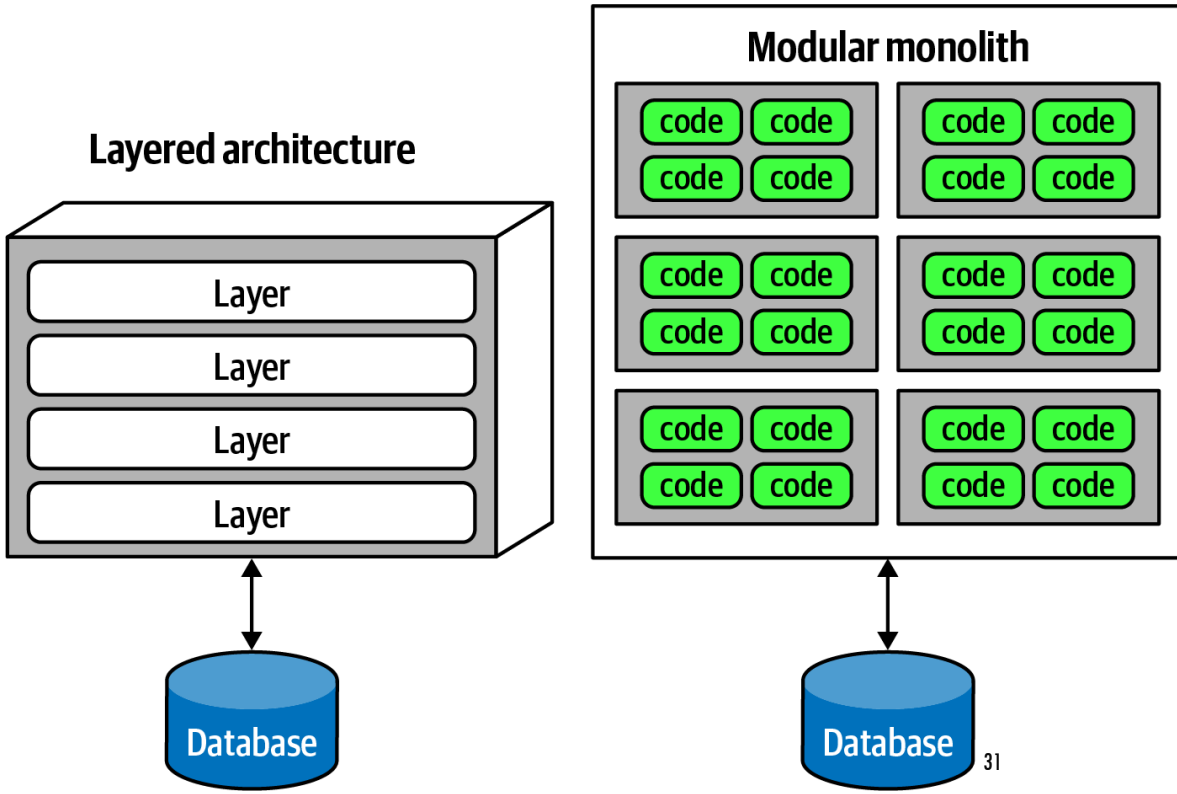


ARCHITECTURE PARTITIONING

The First Law of Software Architecture: Everything in software is a trade-off

Top level partitioning

Ex. monolith types

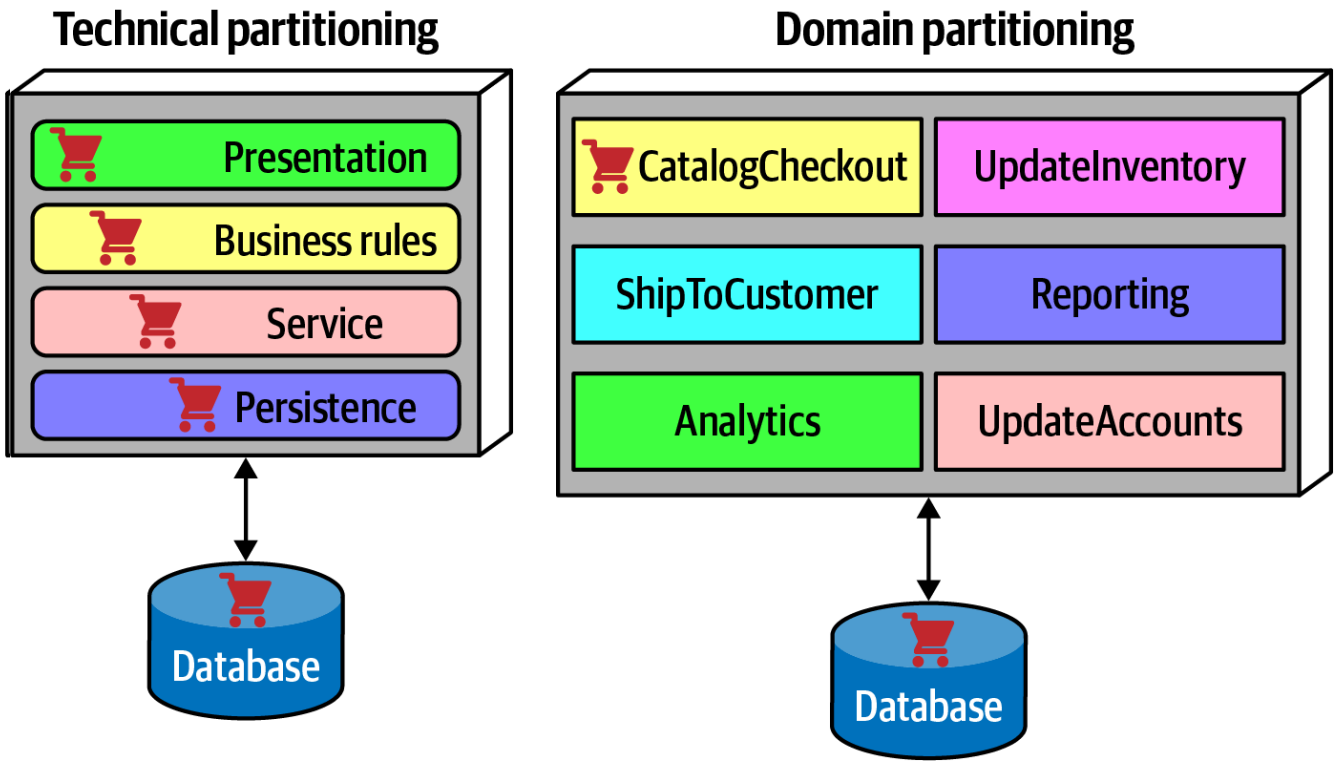


What is the partitioning criterion?

ARCHITECTURE PARTITIONING

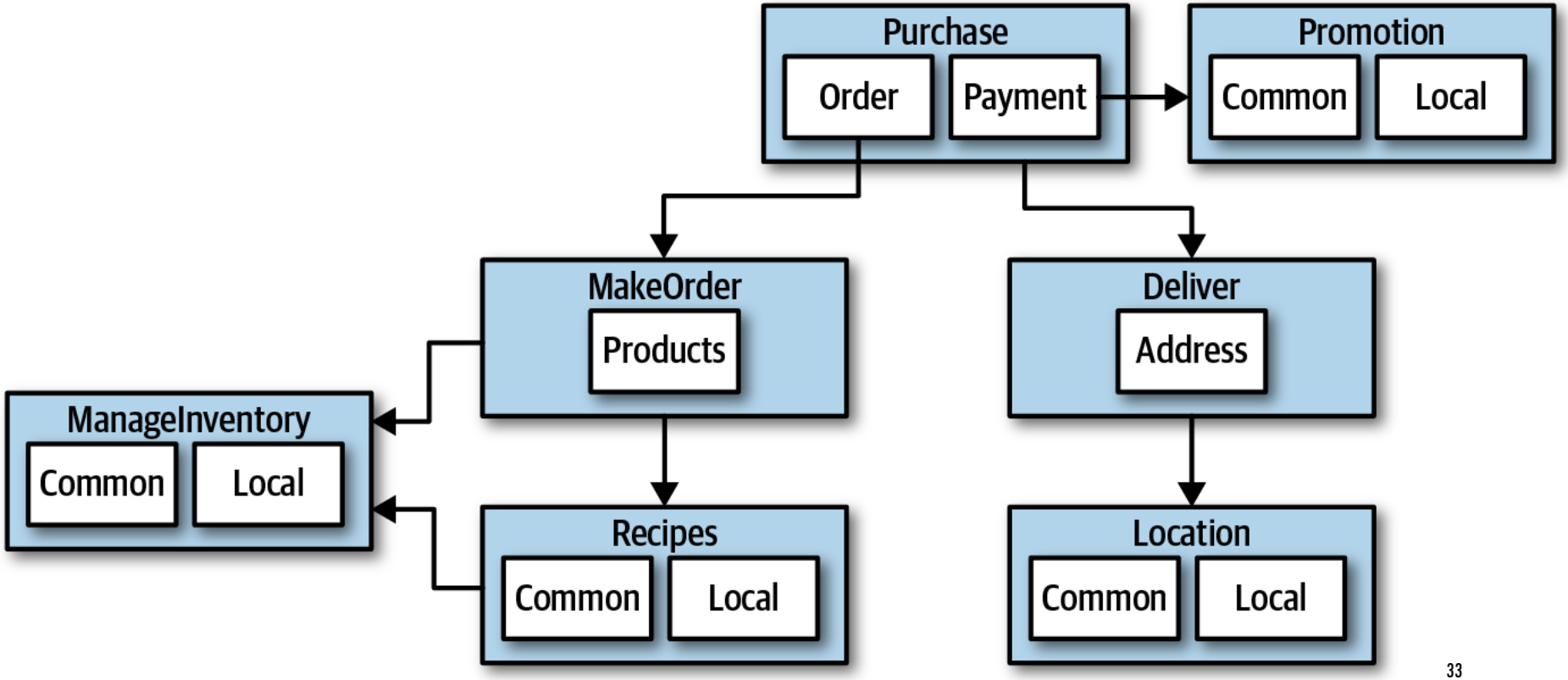
Separation of **technical** concerns vs. separation of **domains**/workflows

Were is the code for CatalogCheckout?



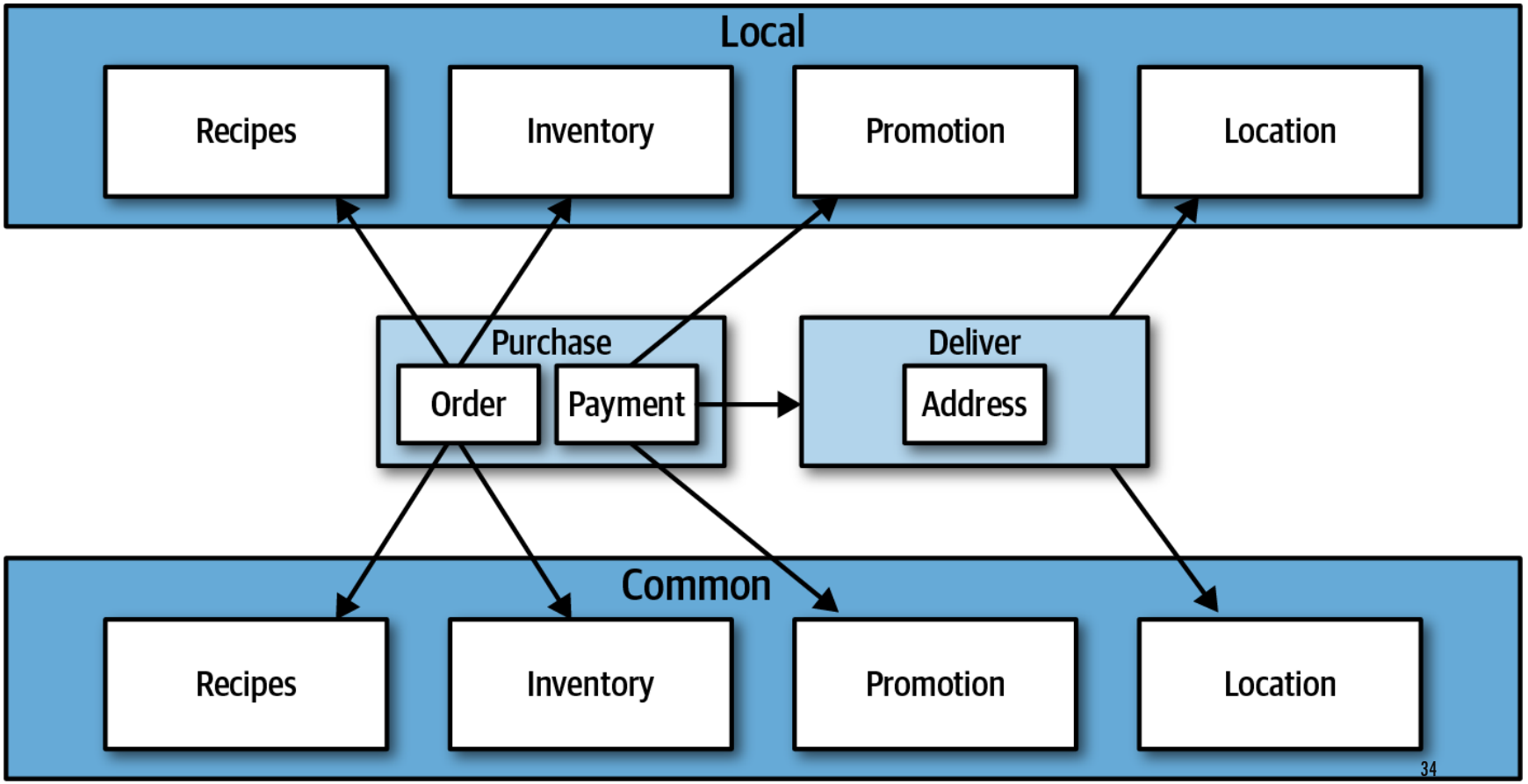
SILICON SANDWICH PARTITIONING

Domain partitioning option



SILICON SANDWICH PARTITIONING

Technical partitioning option



SILICON SANDWICH PARTITIONING DISCUSSION

Domain approach

Advantages

- Modeled more closely toward how the business functions rather than an implementation detail
- Easier to utilize the **Inverse Conway Maneuver** to build cross-functional teams around domains
- Aligns more closely to the modular monolith and microservices architecture styles
- Message flow matches the problem domain
- Easy to migrate data and components to distributed architecture

Disadvantage

- Customization code appears in multiple places

Technical approach

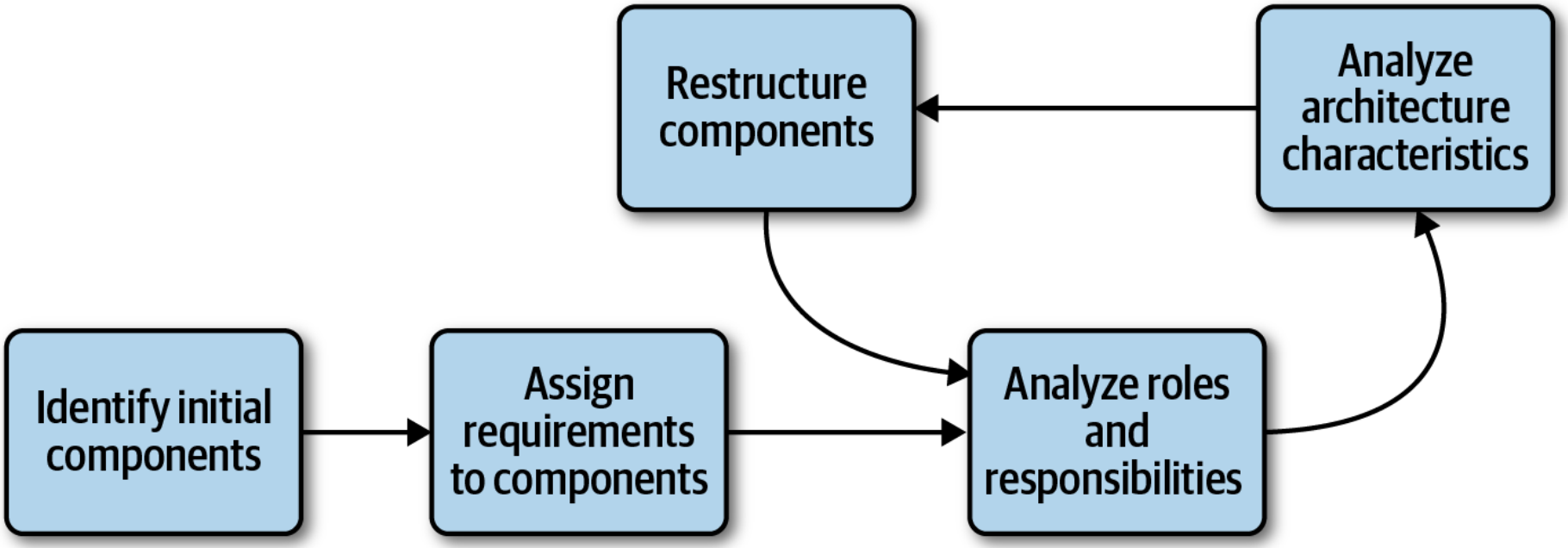
Advantages

- Clearly separates customization code.
- Aligns more closely to the layered architecture pattern.

Disadvantages

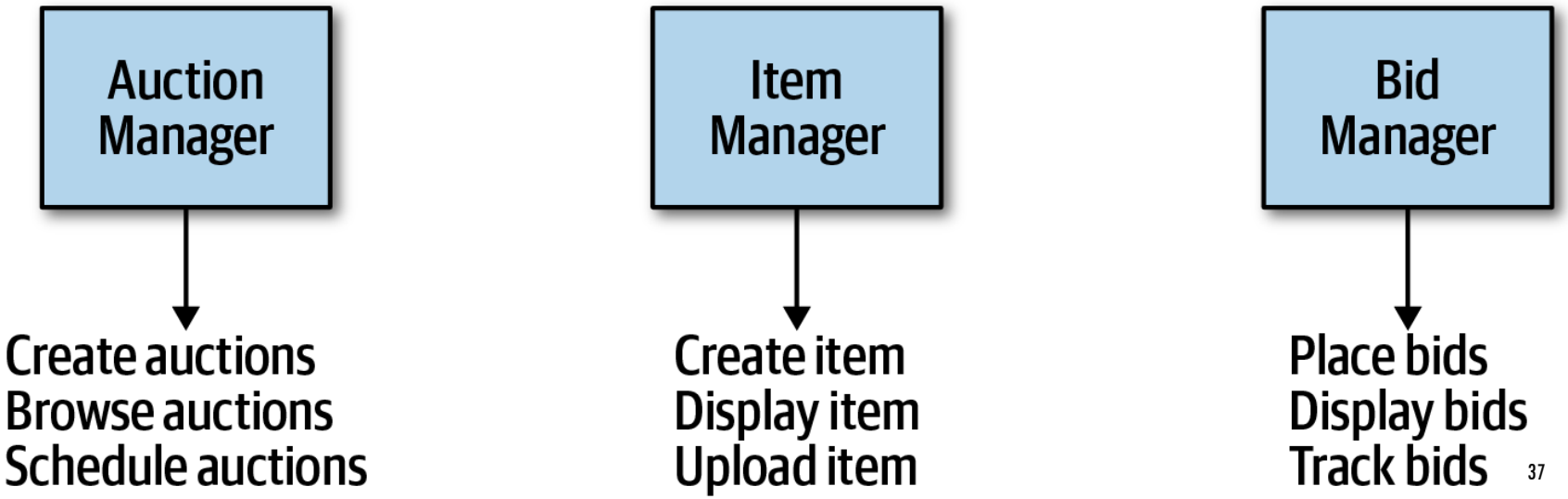
- Higher degree of global coupling. Changes to either the Common or Local component will likely affect all the other components.
- Developers may have to duplicate domain concepts in both common and local layers.
- Typically higher coupling at the data level. In a system like this, the application and data architects would likely collaborate to create a single database, including customization and domains. That in turn creates difficulties in untangling the data relationships if the architects later want to migrate this architecture to a distributed system.

COMPONENT IDENTIFICATION FLOW



COMPONENT DESIGN

- Discovering components: meaningful if you need more than just CRUD operations.
- For CRUD operations you can just use frameworks (ex. NakedObjects, Isis) to build frontends over relational database tables.
- Entity trap anti-pattern



COMPONENT DESIGN

- **Actor/Action driven**
  - works well when the requirements feature distinct roles and the kinds of actions they can perform.
  - works well for all types of systems, monolithic or distributed.
- **Event storming**
  - comes from domain-driven design (DDD) and shares popularity with microservices
  - assumes the project will use messages and/or events to communicate between the various components
  - works well in distributed architectures like microservices that use events and messages
- **Workflow approach**
  - models the components around workflows, much like event storming, but without the explicit constraints of building a message-based system
  - identifies the key roles, determines the kinds of workflows these roles engage in, and builds components around the identified activities

GOING, GOING, GONE KATA

Description

- An auction company wants to take its auctions online to a nationwide scale. Customers choose the auction to participate in, wait until the auction begins, then bid as if they are there in the room with the auctioneer.

Users

- Scale up to hundreds of participants per auction, potentially up to thousands of participants, and as many simultaneous auctions as possible.

Requirements

- Auctions must be (near) real-time.
- Bidders register with a credit card; the system automatically charges the card if the bidder wins.

GOING, GOING, GONE KATA

Requirements

- Participants must be tracked via a reputation index.
- Bidders can see a live video stream of the auction and all bids as they occur.
- Both online and live bids must be received in the order in which they are placed.

Additional context

- Auction company is expanding aggressively by merging with smaller competitors.
- Budget is not constrained. This is a strategic direction.
- Company just exited a lawsuit where it settled a suit alleging fraud.

ACTOR/ACTIONS DECOMPOSITION APPROACH

Roles and actions:

Bidder

- View live video stream, view live bid stream, place a bid

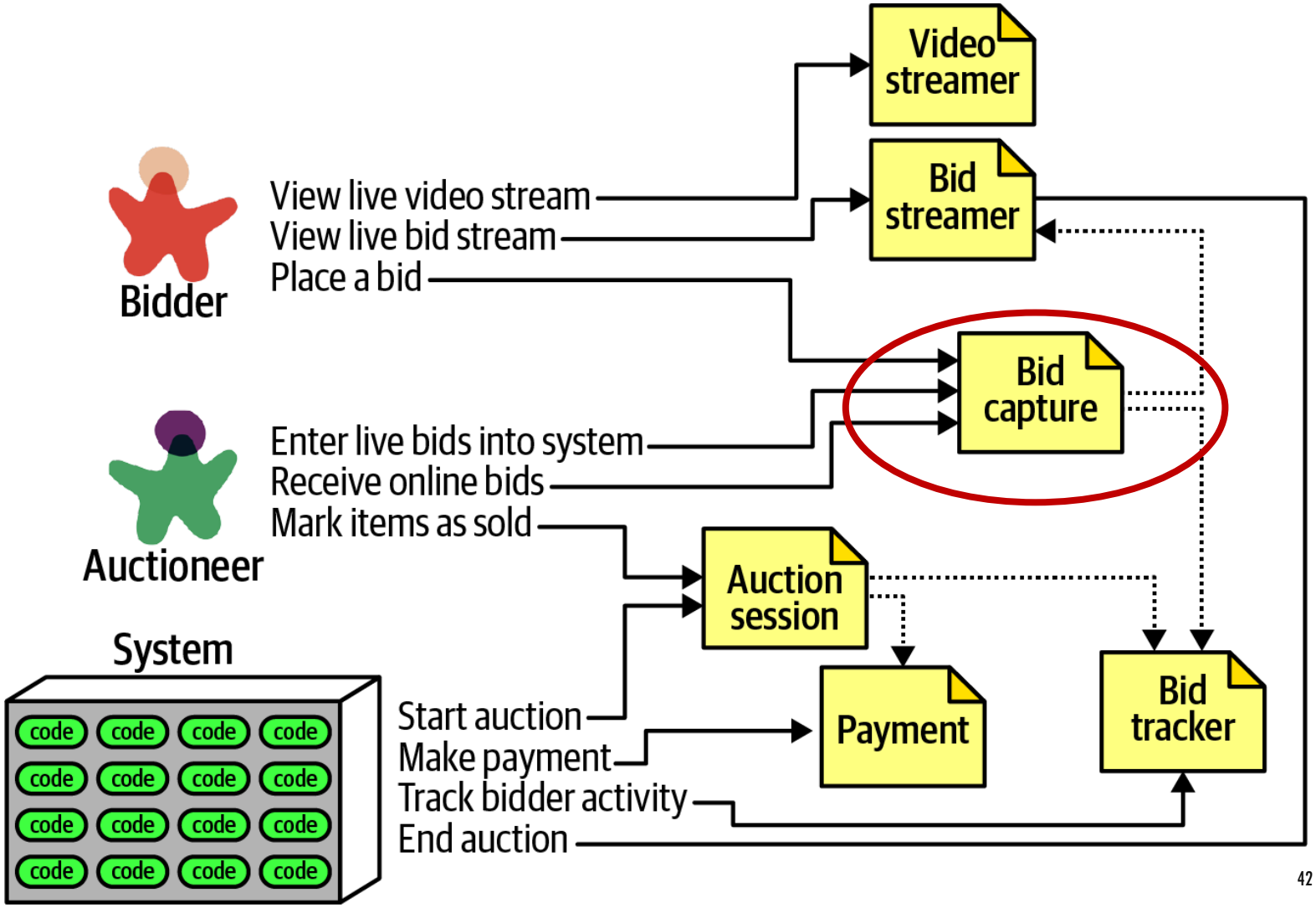
Auctioneer

- Enter live bids into system, receive online bids, mark item as sold

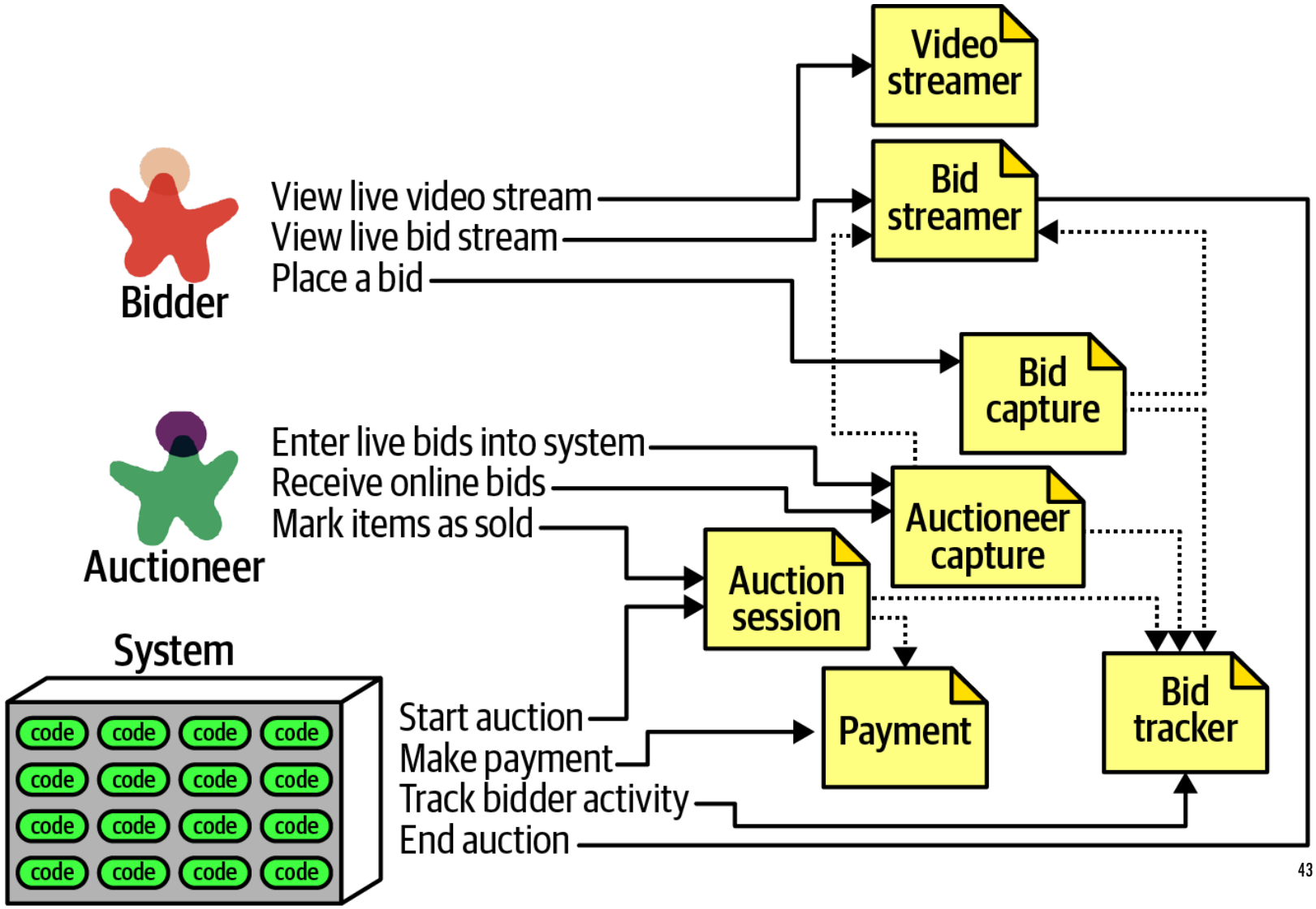
System

- Start auction, make payment, track bidder activity

INITIAL COMPONENTS



SECOND ITERATION



MONOLITHIC VS DISTRIBUTED

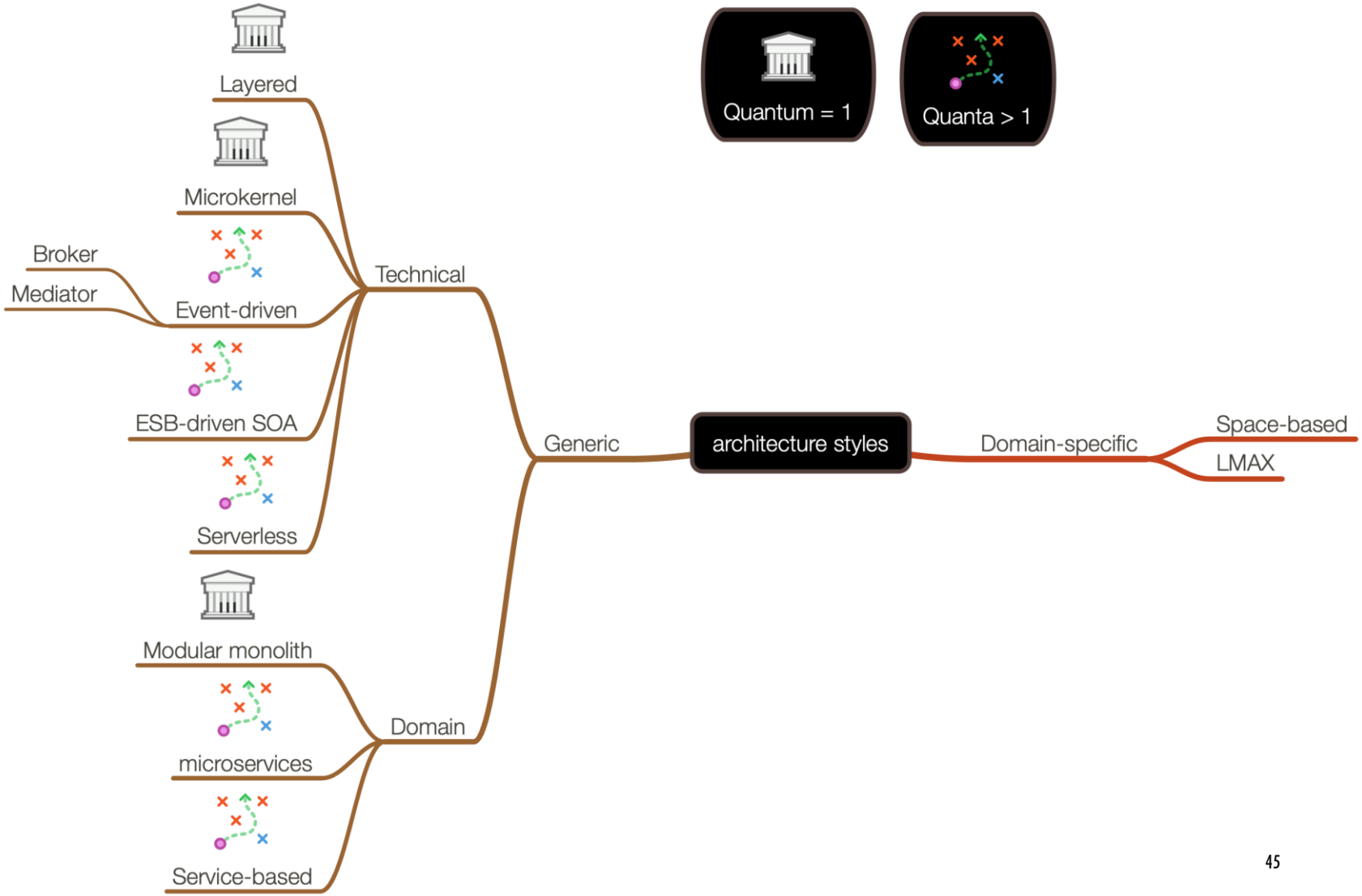
A monolithic architecture typically features a single deployable unit, including all functionality of the system that runs in the process, typically connected to a single database.

A distributed architecture is the opposite—the application consists of multiple services running in their own ecosystem, communicating via networking protocols. Distributed architectures may feature finer-grained deployment models, where each service may have its own release cadence and engineering practices,

Are all the components having the same architecture characteristics?

=> monolithic

WRAP-UP


Quantum = 1
Quanta > 1
architecture styles
Generic
Technical
Layered
Microkernel
Event-driven
Broker
Mediator
ESB-driven SOA
Serverless
Domain
Modular monolith
microservices
Service-based
Domain-specific
Space-based
LMAX

LET'S SWITCH TO MOODLE AND TAKE A QUIZ

It takes 4 minutes.